

Ocelo

Banco de dados

Onde os dados ficam hoje, campo a campo, e a arquitetura sugerida em PostgreSQL para quando o catálogo sair dos arquivos TypeScript: diagrama de relacionamentos, tabelas, índices, consultas, decisões, fluxos de dados, operação e como escalar.

Copyright (©) 2026 Erik Pimenta
Versão online: https://ravelroot.github.io/Network-Protocol-Packet-Explorer/

Versão 1.1.0 · Novembro de 2026 · Todos os direitos reservados

Versão 1.1.0 · Novembro de 2026

Novidades da versão 1.1.0

A versão 1.1.0 muda a aparência do Ocelo e acrescenta um único dado: a **paleta de cores** escolhida por quem usa. Ela entra nas preferências salvas no navegador e, na arquitetura sugerida, no `jsonb` de preferências da conta. Nada muda no catálogo: os 33 protocolos, os passos e os pacotes continuam iguais.

> **Onde está:** o campo novo aparece marcado com NOVO nas tabelas dos capítulos "Contexto" e "Tabelas", e o fluxo de preferências com conta está em "Fluxos de dados".

Onde	O que muda
`ocelo:preferencias`	Campo `paleta`, com `ocelo`, `bento`, `sonda` ou `nexo`; o tema, sem valor salvo, passa a seguir o do sistema
`usuario.preferencias`	Recebe a paleta e ganha a restrição `preferencias_validas`, com os mesmos valores aceitos no navegador
`legenda.cor`	Continua guardando o nome do token; o tom final agora depende da paleta e do tema
Migrações	Novo arquivo `003_preferencias_paleta.sql` com a restrição

Sumário

Contexto 5

- Onde os dados estão hoje 5
- Quando vale migrar 5
- Princípios do modelo 5

Diagrama de relacionamentos 6

- Entidades 6
- Relacionamentos 7

Tabelas 8

- versao 8
- protocolo 8
- protocolo_texto 8
- no, ligacao e faixa 9
- legenda 9
- passo e fato 9
- pacote 10
- usuario e progresso (opcionais) 10

Índices e consultas 11

- Índices 11
- Consultas principais 11

Decisões de arquitetura 12

- PostgreSQL 12
- Textos bilíngues em jsonb 12
- Quadros em jsonb 12
- Cores como tokens 12
- Versões imutáveis 12
- O site continua estático 12

Fluxos de dados 13

- Edição e publicação 13

Leitura no site 13
Preferências com conta 13
Migrando os dados atuais 13

Operação e escala 14

Backup e recuperação 14
Segurança 14
Como escalar 14
Migrações 14

Contexto

Onde os dados estão hoje

Até a versão 1.1.0 o Ocelo não usa banco de dados. Os 33 protocolos são arquivos TypeScript em `src/dados/protocolos`, validados pelos testes e empacotados no build estático. As preferências de cada pessoa ficam no `localStorage` do navegador, na chave `ocelo:preferencias`.

Campo	Valores aceitos	Observação
`idioma`	`pt` ou `en`	Sem valor salvo, segue o idioma do navegador
`tema`	`claro` ou `escuro`	Sem valor salvo, segue o tema do sistema
`paleta` NOVO	`ocelo`, `bento`, `sonda` ou `nexo`	Sem valor salvo, usa a paleta Ocelo
`vista`	`rede` ou `sequencia`	Vista do palco
`velocidade`	0,5, 1, 1,5, 2 ou 3	Velocidade da reprodução

Esse desenho é o certo para o volume atual: o catálogo muda pouco, é público, cabe em 170 kB comprimidos e a hospedagem estática no GitHub Pages não tem custo nem superfície de ataque.

Quando vale migrar

- Edição sem deploy: professores ou uma equipe de conteúdo precisam criar e revisar protocolos por uma interface, com rascunho e publicação.
- Contas e progresso: turmas querem acompanhar quem concluiu cada protocolo e levar as preferências para outros aparelhos.
- Mais idiomas e traduções feitas por pessoas diferentes.
- Catálogo grande (centenas de protocolos ou variações), em que carregar tudo no primeiro acesso deixa de compensar.

> **Recomendação:** enquanto nenhum desses pontos existir, mantenha os arquivos TypeScript. Quando um deles aparecer, siga este documento: o modelo foi desenhado para receber exatamente os dados que já existem, sem perder a validação.

Princípios do modelo

- PostgreSQL como fonte da verdade, com integridade garantida por chaves e restrições, não pela aplicação.
- Conteúdo bilíngue em `jsonb` com as chaves de idioma, validado por restrição.
- A especificação de cada quadro em `jsonb`, porque ela já é validada pelo motor de quadros e muda de forma conforme o protocolo.
- Cores guardadas como nomes de token (`anil`, `oliva`, `ouro`, `brasa`, `neutra`), nunca como valores: o tom final depende da paleta e do tema escolhidos por quem usa.
- Versões publicadas imutáveis: o site lê sempre uma versão fechada, e a edição acontece num rascunho.
- O site continua estático: o banco gera um JSON por versão, servido por CDN.

Diagrama de relacionamentos

Entidades

Em linho, o catálogo versionado; em rosé, os dados de quem usa, que só existem se houver contas. Um protocolo pertence a uma versão, tem textos por idioma, fatos, nós, ligações, faixas, legenda e passos. Cada passo tem fatos e pacotes; cada pacote referencia os nós de origem e destino e um tipo da legenda.

versao	
PK id	bigint
numero	text
estado	enum
publicada_em	timestamptz
autor	text

protocolo	
PK id	text
PK FK versao_id	bigint
sigla	text
categoria	enum
camada	smallint
ordem	smallint

protocolo_texto	
PK FK protocolo_id	text
PK idioma	char(2)
nome	text
resumo	text
descricao	text
observacao	text

no	
PK FK protocolo_id	text
PK id	text
dispositivo	enum
nome	jsonb
ip, mac	inet, macaddr
x, y	numeric

ligacao	
PK FK protocolo_id	text
PK FK origem	text
PK FK destino	text

faixa	
PK FK protocolo_id	text
PK ordem	smallint
FK no_id	text

legenda	
PK FK protocolo_id	text
PK tipo	text
nome	jsonb
cor	enum token

passo	
PK id	bigint
FK protocolo_id	text
ordem	smallint
titulo, resumo	jsonb
descricao	jsonb

fato	
PK id	bigint
FK protocolo_id	text
FK passo_id	bigint
ordem	smallint
rotulo, valor	jsonb

pacote	
PK id	bigint
FK passo_id	bigint
ordem	smallint
FK de, para	text
FK tipo	text
dt_ms, perdido	int, bool
quadro	jsonb

usuario	
PK id	uuid
email	citext
preferencias	jsonb
criado_em	timestamptz

progresso	
PK FK usuario_id	uuid
PK FK protocolo_id	text
ultimo_passo	smallint
concluido_em	timestamptz

Relacionamentos

De	Para	Cardinalidade	Regra
versao	protocolo	1:N	Apagar uma versão publicada é proibido; rascunhos apagam em cascata
protocolo	protocolo_texto	1:N	Um registro por idioma; `pt` e `en` obrigatórios
protocolo	no, ligacao, faixa, legenda	1:N	Em cascata com o protocolo
protocolo	passo	1:N	Ordem única por protocolo
passo	pacote	1:N	Ordem única por passo
passo / protocolo	fato	1:N	Um fato pertence a um passo ou à visão geral, nunca aos dois
pacote	no	N:1 (duas vezes)	Origem e destino precisam existir no mesmo protocolo e estar nas faixas
pacote	legenda	N:1	O tipo precisa existir na legenda do protocolo
usuario	progresso	1:N	Apagar a conta apaga o progresso

Tabelas

versao

Campo	Tipo	Restrições	Descrição
`id`	bigint	PK, identity	Identificador da versão do catálogo
`numero`	text	NOT NULL, UNIQUE	Versão semântica, como 1.1.0
`estado`	enum `versao_estado`	NOT NULL, `rascunho` ou `publicada`	Só rascunhos podem ser editados
`publicada_em`	timestamptz	NULL até publicar	Momento da publicação
`autor`	text	NOT NULL	Quem publicou

protocolo

Campo	Tipo	Restrições	Descrição
`id`	text	PK (com `versao_id`), CHECK (`id ~ '^[a-z0-9-]+$'`)	Usado no endereço `#/id`
`versao_id`	bigint	PK, FK `versao(id)`	Versão a que pertence
`sigla`	text	NOT NULL	Sigla exibida (HTTP, Wi-Fi, TLS/SSL)
`categoria`	enum `categoria`	`core`, `aplicacao` ou `seguranca`	Aba da lista
`camada`	smallint	CHECK (`camada BETWEEN 1 AND 7`)	Camada OSI
`ordem`	smallint	NOT NULL, UNIQUE (`versao_id`, `ordem`)	Posição na lista

protocolo_texto

Campo	Tipo	Restrições	Descrição
`protocolo_id`, `versao_id`	text, bigint	PK, FK `protocolo`	Protocolo
`idioma`	char(2)	PK, CHECK (`idioma IN ('pt','en')`)	Idioma do texto
`nome`, `resumo`, `descricao`	text	NOT NULL, CHECK (`length(trim(x)) > 0`)	Textos da lista e da visão geral
`observacao`	text	NULL	Nota da legenda do fluxo de pacotes

no, ligacao e faixa

Campo	Tipo	Restrições	Descrição
`no.id`	text	PK (com protocolo)	Identificador local, como `cliente`
`no.dispositivo`	enum `dispositivo`	12 valores	Desenho isométrico
`no.nome`	jsonb	CHECK (`nome ? 'pt' AND nome ? 'en'`)	Nome bilíngue
`no.ip` , `no.mac`	inet, macaddr	NULL	Endereços quando existirem
`no.x` , `no.y`	numeric(5,2)	CHECK x 6–94, y 12–88	Posição no palco
`ligacao.origem` , `destino`	text	PK, FK `no` ; CHECK (`origem <> destino`)	Ligação sem direção
`faixa.ordem` , `no_id`	smallint, text	PK (protocolo, ordem); UNIQUE (protocolo, `no_id`)	Ordem das faixas na sequência

legenda

Campo	Tipo	Restrições	Descrição
`tipo`	text	PK (com protocolo)	Chave usada pelos pacotes, como `dns`
`nome`	jsonb	Bilíngue	Texto da legenda
`cor`	enum `cor`	`anil` , `oliva` , `ouro` , `brasa` , `neutra`	Nome do token de cor; cada paleta e cada tema definem o tom final

passo e fato

Campo	Tipo	Restrições	Descrição
`passo.id`	bigint	PK, identity	Identificador
`passo.ordem`	smallint	UNIQUE (protocolo, ordem)	Ordem do passo (3 a 7 por protocolo)
`passo.titulo` , `resumo` , `descricao`	jsonb	Bilíngue	Textos do passo
`fato.protocolo_id` / `passo_id`	text / bigint	CHECK (`num_nonnulls(protocolo_id, passo_id) = 1`)	Dono do fato
`fato.rotulo` , `valor`	jsonb	Bilíngue	Linha da tabela de fatos

pacote

Campo	Tipo	Restrições	Descrição
`id`	bigint	PK, identity	Identificador
`passo_id`, `ordem`	bigint, smallint	FK `passo`; UNIQUE (`passo_id`, `ordem`)	Posição dentro do passo
`de`, `para`	text	FK `no` do mesmo protocolo; CHECK (`de <> para`)	Origem e destino
`proto`	text	NOT NULL	Coluna Protocolo da tabela
`tipo`	text	FK `legenda` do mesmo protocolo	Cor e legenda
`rotulo`, `info`, `resumo`, `nota`	jsonb	Bilíngue; `resumo` e `nota` opcionais	Textos da seta, da tabela e do cartão
`dt_ms`	integer	CHECK (`dt_ms >= 0`)	Intervalo para o pacote anterior
`perdido`	boolean	NOT NULL DEFAULT false	Pacote que não chega
`quadro`	jsonb	NOT NULL, validado por JSON Schema	Especificação do quadro (enlace, rede, portas, flags, carga)

usuario e progresso (opcionais)

Campo	Tipo	Restrições	Descrição
`usuario.id`	uuid	PK DEFAULT `gen_random_uuid()`	Identificador
`usuario.email`	citext	UNIQUE	Login
`usuario.preferencias`	jsonb	Validado por CHECK	Idioma, tema, paleta **NOVO**, vista e velocidade, com os mesmos valores aceitos no navegador
`progresso.ultimo_passo`	smallint	NOT NULL	Último passo visto
`progresso.concluido_em`	timestamptz	NULL	Quando terminou o protocolo

A validação das preferências fica no próprio banco, para que nenhum cliente grave um valor que o site não sabe aplicar:

```
ALTER TABLE usuario ADD CONSTRAINT preferencias_validas CHECK (
  (preferencias ->> 'idioma') IN ('pt', 'en')
  AND (preferencias ->> 'tema') IN ('claro', 'escuro')
  AND coalesce(preferencias ->> 'paleta', 'ocelo') IN ('ocelo', 'bento', 'sonda', 'nexo')
  AND (preferencias ->> 'vista') IN ('rede', 'sequencia')
  AND (preferencias ->> 'velocidade')::numeric IN (0.5, 1, 1.5, 2, 3)
);
```

Índices e consultas

Índices

Índice	Para quê
`protocolo (versao_id, ordem)`	Montar a lista na ordem certa
`protocolo (versao_id, categoria, camada)`	Contagens das abas e filtro por camada
`passo (protocolo_id, versao_id, ordem)`	Ler os passos de um protocolo
`pacote (passo_id, ordem)`	Ler os pacotes de um passo
`protocolo_texto` GIN (`to_tsvector(nome || resumo)`)	Busca por texto no editor de conteúdo
`progresso (usuario_id)`	Painel de progresso da pessoa

Consultas principais

```
-- catálogo publicado, na ordem da lista
SELECT p.id, p.sigla, p.categoria, p.camada, t.nome, t.resumo
FROM protocolo p
JOIN protocolo_texto t
  ON t.protocolo_id = p.id AND t.versao_id = p.versao_id AND t.idioma = $1
WHERE p.versao_id = (SELECT id FROM versao WHERE estado = 'publicada'
                     ORDER BY publicada_em DESC LIMIT 1)
ORDER BY p.ordem;

-- passos e pacotes de um protocolo
SELECT s.ordem AS passo, k.ordem, k.de, k.para, k.proto, k.tipo, k.dt_ms, k.perdido, k.quadro
FROM passo s JOIN pacote k ON k.passo_id = s.id
WHERE s.protocolo_id = $1 AND s.versao_id = $2
ORDER BY s.ordem, k.ordem;

-- preferências da pessoa, já no formato do localStorage
SELECT preferencias FROM usuario WHERE id = $1;
```

Decisões de arquitetura

PostgreSQL

Relacional, com chaves compostas, restrições `CHECK`, `jsonb` indexável, tipos de rede nativos (`inet` e `macaddr`) e maturidade de backup e réplica. Um SQLite serviria para um editor local, mas o PostgreSQL atende o editor e a publicação com o mesmo modelo.

Textos bilíngues em jsonb

Cada texto curto vira um objeto `{"pt": ..., "en": ...}` com restrição que exige as duas chaves. Os textos longos do protocolo ficam em `protocolo_texto`, uma linha por idioma, o que facilita revisar e acrescentar idiomas sem mudar o esquema.

Quadros em jsonb

A especificação de cada quadro varia muito (TCP, UDP, ICMP, 802.11, EAPOL, campos da carga). Normalizar tudo em tabelas criaria dezenas de tabelas raramente consultadas. Guardar em `jsonb` validado por JSON Schema mantém a flexibilidade, e o motor de quadros continua sendo quem monta e confere os bytes.

Cores como tokens

A legenda guarda o nome da cor, e não um valor hexadecimal. Assim a mesma linha do banco aparece certa nas quatro paletas e nos dois temas, e uma paleta nova não exige migração de dados: basta o CSS definir os cinco tokens para ela.

Versões imutáveis

Editar sempre um rascunho e publicar uma versão fechada garante que o site nunca mostra conteúdo pela metade e permite voltar para a versão anterior em segundos. Cada versão publicada gera um arquivo `catalogo-<versao>.json`.

O site continua estático

O navegador não consulta o banco. Na publicação, um job exporta a versão para JSON, roda a mesma validação dos testes atuais e envia o arquivo para a CDN junto com o build. Assim a leitura escala sem custo e o banco fica protegido atrás da área de edição.

Fluxos de dados

Edição e publicação

1. A pessoa editora cria uma versão em rascunho copiando a última publicada.
2. Edita protocolos, passos e pacotes na área de edição; cada gravação passa pelas restrições do banco.
3. Pede a publicação. Um job exporta o rascunho para JSON e roda a validação completa (a mesma de `validacao.test.ts`) e a montagem de todos os quadros.
4. Se tudo passar, a versão vira `publicada`, o JSON vai para a CDN e o site passa a apontar para ele.
5. Se algo falhar, o rascunho continua editável e o relatório mostra o protocolo e o item com problema.

Leitura no site

O site baixa o índice do catálogo (siglas, nomes, categorias e camadas) e, ao abrir um protocolo, o arquivo daquele protocolo. Com cache longo e nomes de arquivo por versão, cada pessoa baixa cada protocolo uma vez só.

Preferências com conta

1. Sem conta, nada muda: o `aparencia.js` e a loja leem e gravam `ocelo:preferencias` no navegador.
2. Ao entrar, o navegador envia as preferências locais; o servidor valida pelo mesmo `CHECK` e guarda em `usuario.preferencias`.
3. Em outro aparelho, a resposta do login devolve as preferências, que o site grava no `localStorage` antes de aplicar o tema e a paleta.

Migrando os dados atuais

1. Exporte os 33 protocolos para JSON a partir dos arquivos TypeScript (o mesmo formato do tipo `Protocolo`).
2. Crie a versão 1.1.0 e importe o JSON numa transação, tabela por tabela, respeitando a ordem das chaves estrangeiras.
3. Confira as contagens: 33 protocolos, 177 passos e 432 pacotes.
4. Gere o JSON publicado e compare com o original byte a byte.

Operação e escala

Backup e recuperação

- `pg_dump` diário em formato custom, guardado fora do servidor, com retenção de 30 dias.
- WAL contínuo para recuperação a um ponto no tempo, se houver edição intensa.
- Os JSON publicados já são um backup do conteúdo e podem reconstruir o catálogo.

Segurança

- Papel só de leitura para o job de exportação e papel de escrita só para a área de edição.
- Row Level Security em `progresso` e `usuario`: cada pessoa só lê e grava o que é dela.
- Conexões com TLS e segredos fora do repositório (variáveis de ambiente ou cofre).
- Auditoria de quem publicou cada versão na tabela `versao`.

Como escalar

Cenário	Estratégia
Mais acessos ao site	Nada muda no banco: a leitura é estática pela CDN
Mais protocolos	Arquivos por protocolo e carregamento sob demanda; índices já cobrem as consultas
Mais idiomas	Novas linhas em `protocolo_texto` e novas chaves nos textos `jsonb`, com a restrição ampliada
Mais paletas	Nenhuma mudança de esquema na legenda; só a lista aceita em `preferencias_validas`
Muitas contas e turmas	Particionar `progresso` por mês, réplicas de leitura para relatórios

Migrações

Cada mudança de esquema vira um arquivo SQL numerado (`001_esquema.sql`, `002_indices.sql`, `003_preferencias_paleta.sql`...), aplicado em ordem dentro de uma transação e registrado numa tabela de controle.